Go2 の強化学習歩行と疑似 LiDAR による巡回・障害物回避

― Genesis 上での 3 層構成(歩行・反応的回避・地図による経路計画)、3D LiDAR への拡張と定量評価 ―

Physical AI 2026 応用編1 最終課題 説明資料 ／ Omnicampus アカウント ID: **3164** ／ コード: `https://github.com/raycast-trail/go2-genesis-patrol` ／ 動画: `https://youtu.be/uhSM8Q9onhM`(YouTube「PAI最終課題_3164」、2 分 49 秒、一部 2 倍速)

1. 概要

四足歩行ロボット Unitree Go2 を物理シミュレータ Genesis 上で扱い、(1) 速度指令に追従する歩行方策を強化学習(PPO、4096 環境並列)で獲得し、(2) その上に、胴体に取り付けた疑似 LiDAR だけを手がかりに障害物を避けながら 2 m 角の正方形経路を巡回するナビゲーション層を載せた。ナビゲーション層の中心は、LiDAR で描いたマス目の地図(占有格子地図)の上で A*(エースター)探索により最短経路を引き直す**地図による経路計画(v3)**である。障害物をランダムに配置した **45 配置**で評価した結果、巡回成功率は回避なしの **9% から 100%** に、障害物への物理接触率は **91% から 0%** になった(同一配置での対応あり McNemar 正確検定 $p < 0.0001$、転倒 0 件)。地図の効果を切り分けるため、地図を持たない反応的回避 2 種(v1・v2、成功率 91%・96%)と比べると、v3 は成功分の所要時間が中央値で 3.4 s・6.1 s 短く(符号検定 $p < 0.0001$)、反応的回避の弱点(正面の障害物での振動、遠回り)をともに解消した。さらに、2D スキャンの水平面より低い箱(0.15 m)とテーブルの天板という「高さ方向で通行可否が変わる障害物」を混ぜた 30 配置では、2D LiDAR の成功率は **3%**(接触 97%)に落ち、点群を胴体の高さ帯で切って 2D 地図に投影する **3D 疑似 LiDAR(72 × 16 層)で 100%・接触 0%** に回復した($p < 0.0001$)。評価は物理接触を主指標とし、Wilson 信頼区間と対応あり検定、シミュレータの非決定性の切り分けを含めて設計した。実機 Go2 の「自己位置 → 地図・経路計画 → 速度指令 → 歩行コントローラ」と同じ層構造を再現しており、歩行(学習)とナビゲーション(決定的・説明可能)を疎結合に分けている点が特徴である。全工程は M1 Max の Mac 1 台(Metal / CPU)で完結し、GPU サーバやクラウドは使っていない。

2. 目的と背景

実機の自律巡回ロボットは「自己位置推定 → 経路計画 → 速度指令 → 歩行コントローラ」の層構造で作られ、下層ほどロボット固有、上層ほどタスク固有である。本課題では、このうち**速度指令 → 歩行**を強化学習で、**センサ → 速度指令**を古典的な制御(反応的回避、最終的には占有格子地図 + A*)で構成し、シミュレータ上で同じ構造を再現することを目的とした。狙いは次の 3 点である。

- **AI の活用**: 歩行は、ニューラルネットワークの方策を強化学習(PPO、4096 環境並列)で学習し、巡回中は毎ステップこの方策の推論で 12 関節を動かす(§4)。ナビゲーション層は、失敗の原因を追えるよう、あえて学習を使わない決定的な規則とした。
- **タスク非依存の歩行方策**: 方策は「与えられた (v_x, v_y, ω_z) に追従する」ことだけを学ぶ。巡回・回避は上層で差し替え可能にする。
- **センサの生データだけで避ける**: 障害物の座標は制御に使わず、LiDAR の距離配列のみを入力とする(障害物座標はシーン生成と採点にのみ使用)。v3 の地図も LiDAR から逐次構築する。3D 版では点群の高さで障害物を判定する。
- **定量評価**: 1 回の成功動画ではなく、配置をランダム化した多数のシーンで成功率・接触率・所要時間・経路効率を測り、改良の効果を対応あり検定で判定する。

3. システム構成

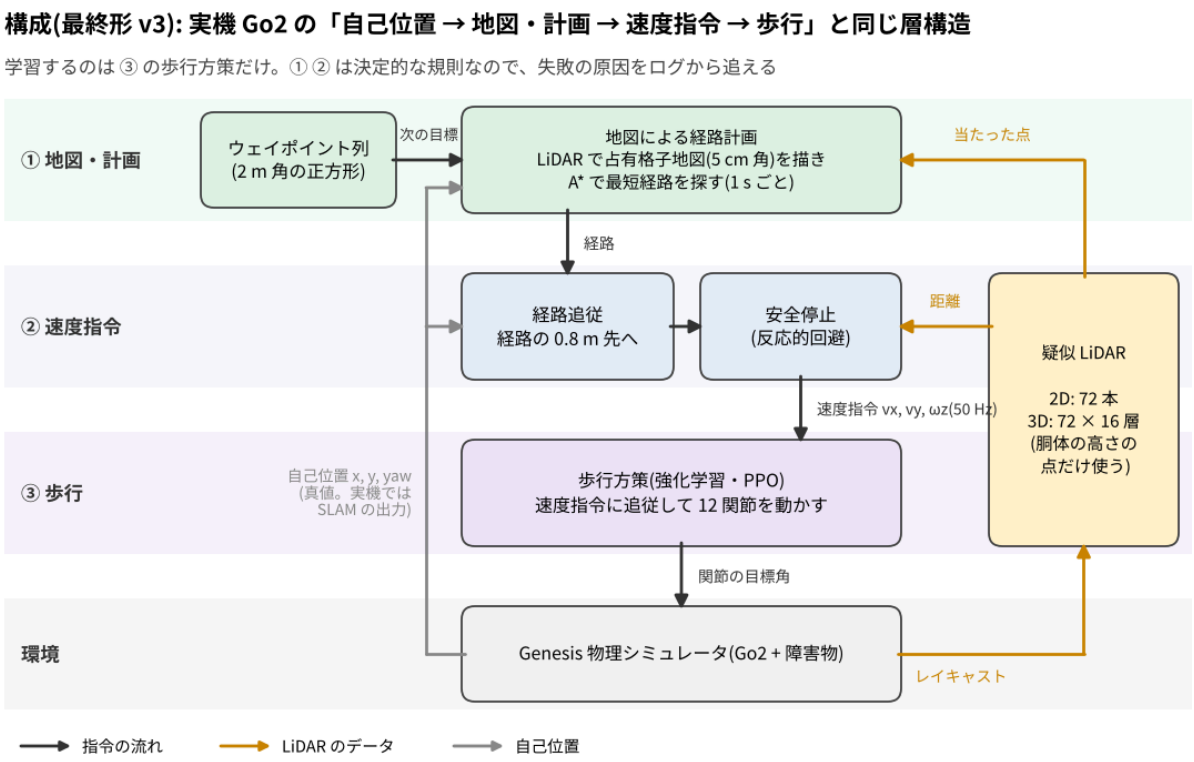


図 1: システム構成(最終形 v3)。上から ① 地図・計画、② 速度指令、③ 歩行の層で、学習するのは ③ の歩行方策だけ。① は LiDAR で描いた地図の上で経路を引き(しくみは図 2)、② はその経路をたどる速度指令を作り、障害物が近すぎれば安全停止する。地図を持たない比較対象(v1・v2)との違いは §8 の図 9 を参照。

層	実装	実機 Go2 との対応
経路追従	4 つのウェイポイント(2 m 角)への P 制御。進行方向誤差 → ω_z、距離 → v_x。到達判定 0.3 m	Nav2 のローカル追従に相当
地図による経路計画(v3)	地面除去後の LiDAR ヒット点から log-odds 占有格子(0.05 m、120×120)を逐次更新。占有セルを Go2 の半径 0.35 m で膨張し、8 近傍 A* で次のウェイポイントまでの経路を 1 s ごとに再計画。経路上 0.8 m 先をローカル目標として経路追従に渡す	Nav2 の costmap + global planner
障害物回避(v1 / v2、v3 の下層)	進路幅 ±0.4 m の「回廊」内に入った LiDAR ヒットのみを正面障害物とみなし、1.0 m 未満で減速、0.45 m 未満で停止・その場旋回。旋回方向は最近接ヒットの反対側。ω_z の変化率を ±0.1 rad/s/step に制限	反応的なローカル回避(DWA/VFH の簡易版)
疑似 LiDAR(2D / 3D)	Genesis の Raycaster を胴体リンクに取付(高さ +0.15 m、床から 0.44 m)。2D: 360°/5° 刻み 72 本、最大 5 m、ヒット点の z < 5 cm を床とみなして除去(地面除去)。3D: 72 方位 × 16 層(垂直 ±30°)の点群を「地面 / 胴体の高さ帯(0.05〜0.49 m)/ 頭上 / 自機」に分類し、胴体帯の点の最小水平距離を 72 本に投影(§9)。12 セクタの最小距離に集約	2D は 3D LiDAR の水平断面。3D の高さ帯投影は Nav2 の voxel layer と同じ処理
歩行方策	PPO(rsl-rl)。観測 45 次元(胴体角速度・重力方向・速度指令・関節角/角速度・前回行動)、行動 12 次元(関節目標角の補正)、50 Hz、PD 制御(k_p=20, k_d=0.5)	Go2 の歩行コントローラ
シミュレータ	Genesis 1.4.4(Apache-2.0)。学習は Metal バックエンド(4096 環境)、評価・録画は CPU バックエンド(1 環境)	—

速度指令の範囲は $v_x \in [0, 0.8]$ m/s、$v_y \in [-0.2, 0.2]$ m/s、$\omega_z \in [-0.8, 0.8]$ rad/s で、回避則の出力もこの範囲にクリップする。衝突判定は Genesis の接触情報(`get_contacts`)を用いる。

v3 の地図と経路計画のしくみ

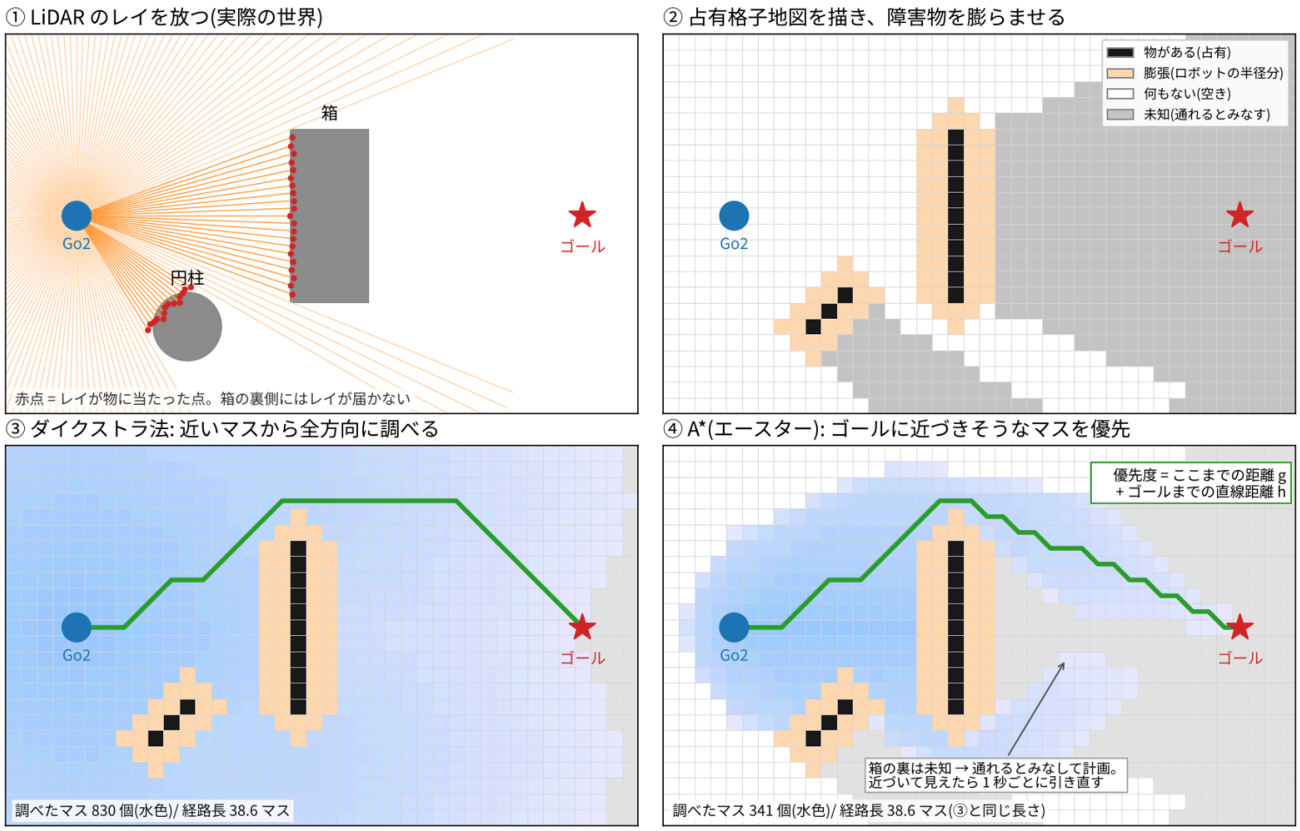


図 2: 占有格子地図と A* 探索のしくみ(説明用の模式図。小さなマス目で実際に計算して作成しており、マスの大きさ・レイの本数は実装と異なる)。

v3 は「歩きながら地図を描き、その地図の上で経路を引き直す」構成で、カーナビの経路探索と同じ考え方である(図 2)。① 床を小さなマス(実装では 5 cm 角)に区切り、LiDAR のレイが物に当たった点のマスを「占有」、レイが通り抜けたマスを「空き」とする。レイが届かないマス(箱の裏など)は「未知」のまま残る。実装では 1 回の観測で決めつけず、マスごとに確からしさ(log-odds)を足し引きし、何度も当たったマスだけを占有とする。これを**占有格子地図**と呼ぶ。② ロボットは点ではなく半径 0.35 m の大きさがあるので、占有マスをその半径だけ膨らませる(膨張)。こうすると「ロボットの中心が膨張領域に入らなければぶつからない」という、点の経路探索の問題に置き換えられる。③④ その地図の上で、スタートからゴールまでの最短経路を探す。ダイクストラ法はスタートから近い順に全方向へマスを調べるのに対し、**A*(エースター)**は「ここまでの距離 g + ゴールまでの直線距離 h」が小さいマスから調べるため、ゴールの方向へ優先して探索が進む。h が実際の残り距離を超えない限り、得られる経路は最短のまま保証される(図の例では調べたマスが 830 個から 341 個に減り、経路の長さは同じ)。未知のマスは通れるとみなして計画し、近づいて見えたら 1 秒ごとに経路を引き直す。Go2 は経路上 0.8 m 先の点を一時的な目標として追従し、反応的回避(v1)は不意の接近に備えた安全停止として下層に残す。

4. 歩行方策の学習(強化学習)

本システムで AI(ニューラルネットワークの学習と推論)を使っているのはこの層である。四足歩行では 12 個の関節を、接地とバランスを保ちながら協調して動かす必要があり、その動かし方を規則として手で書くのは難しい。そこで、シミュレータの中でロボットに試行錯誤させて歩き方を身につけさせる**強化学習**を用いた。強化学習では、状態を見て行動を選ぶ関数(**方策**)が実際に行動し、その結果に応じて**報酬**を受け取る。方策

は、将来にわたって受け取る報酬の合計が大きくなるように少しずつ修正される。正解の動きを教える必要はなく、「何が良い結果か」を報酬で伝えるだけでよい(図 3)。

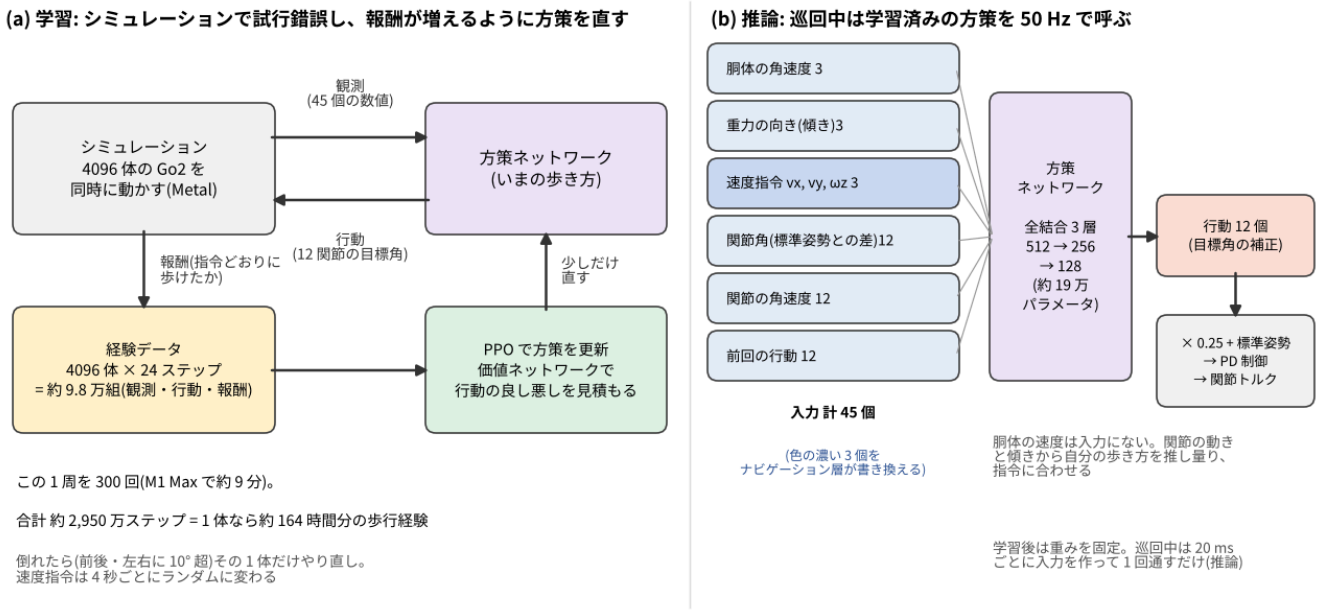


図 3: 強化学習による歩行方策のしくみ。(a) 学習: 4096 体の Go2 を同時にシミュレーションし、集めた経験から PPO で方策を少しずつ更新する。(b) 推論: 巡回中は学習済みのネットワークに 20 ms ごとに観測を入れ、12 関節の目標角を得る。ナビゲーション層が書き換えるのは入力のうち速度指令の 3 個だけ。

入力と出力

方策は全結合 3 層(512・256・128 ユニット、活性化関数 ELU、約 19 万パラメータ)の小さなネットワークである。入力は 45 個の数値で、胴体の角速度(3)、重力の向き = 胴体の傾き(3)、**速度指令 v_x, v_y, ω_z(3)**、関節角の標準姿勢からの差(12)、関節の角速度(12)、前回の行動(12)からなる。胴体の速度そのものは入力に含まれず、方策は関節の動きと傾きから自分の歩き方を推し量って指令に合わせる。出力は 12 関節の目標角の補正量で、0.25 倍して標準姿勢に足した角度を PD 制御($k_p = 20$、$k_d = 0.5$)で追わせ、関節トルクにする。実機の制御遅れを模して、行動は 1 ステップ遅れて反映される。**速度指令が入力の一部になっていることが、ナビゲーション層と歩行層の境目**である。ナビゲーション層はこの 3 個を書き換えるだけで、歩行方策の中身には触れない。

報酬の設計

報酬の項目	何を評価するか	重み
速度の追従(前後・左右)	指令した v_x, v_y と実際の速度の差が小さいほど大きい($\exp(-\text{誤差}^2 / 0.25)$)	+1.0
旋回の追従	指令した ω_z と実際の旋回速度の差が小さいほど大きい	+0.2
上下動	胴体の上下方向の速度の 2 乗(跳ねる歩き方を抑える)	−1.0
胴体の高さ	目標の高さ 0.3 m からのずれの 2 乗	−50
行動の変化	前回の行動との差の 2 乗(関節をがたつかせない)	−0.005
標準姿勢からのずれ	関節角の標準姿勢からの差の絶対値の合計(不自然な姿勢を抑える)	−0.1

各項にシミュレーションの刻み幅 0.02 s を掛けて毎ステップ合計する。前後・左右に 10° を超えて傾いたら転倒とみなしてその 1 体だけ最初からやり直し、1 回の試行は最長 20 s。速度指令は 4 s ごとに範囲内でランダムに選び直すので、方策は指令が途中で変わる状況にも慣れる。

PPO と並列学習

方策の更新には **PPO**(Proximal Policy Optimization)を用いた。方策と同じ形のもう 1 つのネットワーク(**価値ネットワーク**)が「この状態からなら、今後どれだけ報酬が得られそうか」を見積もり、実際に得た報酬がその見積もりより良かった行動を増やし、悪かった行動を減らす向きに方策を直す(割引率 $\gamma = 0.99$、GAE の $\lambda = 0.95$)。PPO の特徴は、1 回の更新で方策が変わりすぎないよう変化の幅を制限すること(クリップ幅 0.2、学習率 0.001 を方策の変化量に応じて自動調整)で、これにより学習が安定する。学習中は行動にランダムなばらつきを加えていろいろな動きを試させ(探索)、巡回時はばらつきを外した出力を使う。

強化学習は大量の試行を必要とするが、Genesis では 4096 体の Go2 を GPU(Apple Silicon の Metal)上で同時に動かせる。1 回の更新では 4096 体 × 24 ステップ = 約 9.8 万組の経験を集め、それを 4 分割して 5 周学習する。これを **300 回**繰り返した(M1 Max で約 9 分、7.4〜7.8 万 steps/s)。合計約 2,950 万ステップで、1 体のロボットなら約 164 時間かかる量の経験を 9 分で得たことになる。

指令範囲の拡張と結果

出発点は Genesis 付属の Go2 歩行例題(Legged Gym 由来の上記の報酬)である。ただし既定の例題は速度指令が **前進 0.5 m/s に固定**されており、「前進しかできない方策」しか得られない。巡回には減速・停止・旋回が必要なので、指令の範囲を $v_x \in [0, 0.8]$ m/s、$v_y \in [-0.2, 0.2]$ m/s、$\omega_z \in [-0.8, 0.8]$ rad/s に広げて学習し直した。

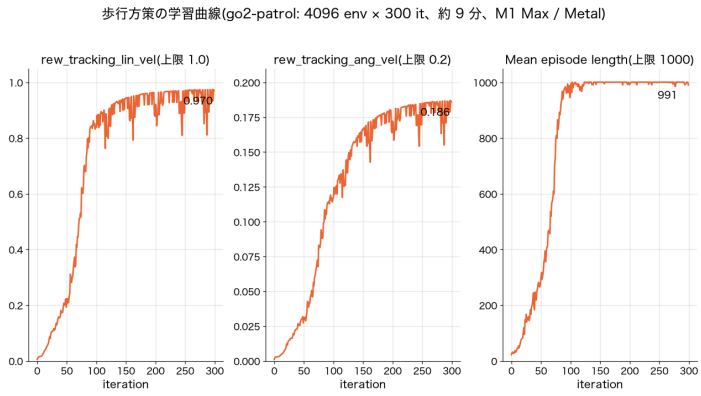


図 4: 学習曲線。速度追従報酬 0.97/1.0、旋回追従 0.186/0.2、エピソード長 991/1000(転倒ほぼゼロ)。

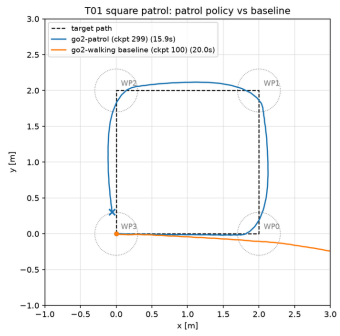


図 5: 正方形巡回(障害物なし)。拡張方策(青)は 4 隅を 15.9 s で回る。既定方策(橙)は旋回指令を無視して直進し 1/4 で時間切れ。

図 5 のとおり、既定方策は WP0 到達後に旋回指令に追従できず直進する。**巡回には指令範囲を広げた再学習が必須**であり、これは後述の 20 配置評価でも既定方策が全条件で成功 0% だったことで裏付けられた。

5. 疑似 LiDAR と反応的回避

Raycaster を既知配置(正面 1.3 m に箱、右 1.85 m に円柱)で単体検証し、距離が幾何計算の期待値と 2 mm 以内で一致すること、Metal と CPU で値が完全一致することを確認した。重要だった落とし穴が 2 つある。(1) 胴体が前傾すると水平レイが床に当たり、2〜5 m 先に偽の障害物が見える。ヒット点の高さで床を除去する処理を入れ、障害物なしの巡回 802 ステップで除去後の偽ヒットが 0 本になることを確認した。(2) 初期姿勢(落下前)はセンサが高さ 0.5 m の箱を越えて見えないため、障害物の高さは 0.6 m 以上とし、着地後からスキャンを使う。

回避則は当初「正面 ±45° と左右セクタの空き具合で旋回方向を決める」設計だったが、3 配置の試験で 2 配置が未到達となった。ログを調べると、「正面右の障害物に対して、左の遠くに別の障害物があるという理由で右(障害物側)へ旋回する」欠陥があった。そこで正面判定を **進路幅 ±0.4 m の回廊**に、旋回方向を **最近接ヒットの反対側**に変更した(v1)。これにより 3 配置すべてで 4/4 到達・接触 0 となった。

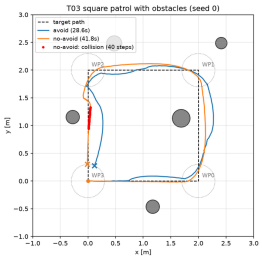


図 6: 障害物あり配置(seed 0)。回避あり(青)は経路脇の円柱を内側に避ける。回避なし(橙)は同じ円柱に 25 秒押し付けられる(赤点)。

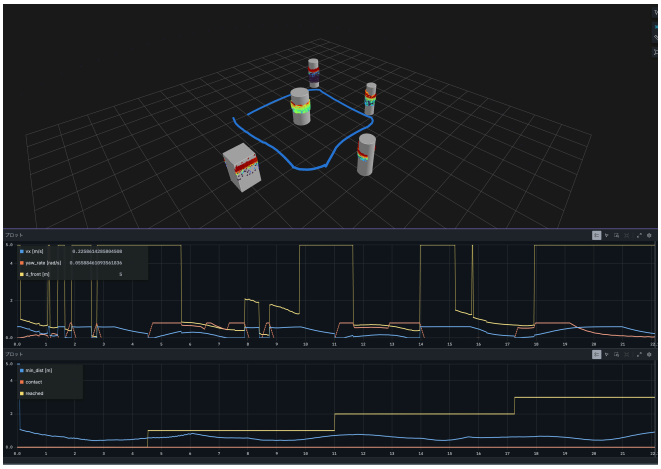


図 7: 巡回ログ(MCAP)を Foxglove で可視化。地面除去後の LiDAR 点群(z で色付け)が 5 つの障害物の側面をなぞり、青い軌跡がその間を通る。下段は v_x・ω_z・正面距離の時系列。

巡回中の全データ(TF、スキャン、点群、軌跡、障害物、指令)は Foxglove SDK で MCAP に書き出せるようにした(ROS 不要)。図 7 は実機の LiDAR 運用と同じ可視化経路で、点群が障害物の輪郭を描き、正面距離が下がると v_x が落ちて ω_z が立つ因果が時系列で読める。

6. 評価設計

- **シーン**: 2 m 角の正方形経路の周囲 3 m 角に、箱(0.3〜0.5 m 角)と円柱(半径 0.1〜0.2 m)を 3〜6 個乱数配置(seed で再現可能)。ウェイポイントから 0.5 m、初期位置から 1.0 m、障害物同士 1.2 m のクリアランスを棄却サンプリングで確保。
- **指標**: 成功 = 60 秒以内に 4 ウェイポイント到達 かつ 転倒なし かつ **物理接触なし**。接触率、ニアミス率(LiDAR 最小距離 < 0.25 m)、成功分の所要時間、経路効率(直線距離 8.0 m / 走行距離)。
- **衝突判定の変更**: 設計当初は LiDAR 最小距離 < 0.25 m を衝突としていたが、正面から押し付けている状態では胴体前端が当たっていてもセンサ中心から 0.3 m 以上あり検出できないことが分かった(横擦りは検出できる)。そこで物理接触を主指標、LiDAR 判定を副指標に改めた。
- **比較**: 同一 seed に対して条件を変える対応あり比較。McNemar 正確検定(成功・接触)、Wilson 95% 信頼区間、所要時間は符号検定・Wilcoxon 符号順位検定(数式を自前実装)。
- **非決定性**: 同一条件でも結果が揺れること(例: ある配置で到達数が 1/4〜4/4)を確認したため、録画の有無・MCAP 出力の有無・プロセスの別など 5 条件 × 3 回で切り分けた。条件間に系統差はなく、CPU バックエンドの真の非決定性と判断した。評価は配置数で平均化する設計のため結論には影響しないが、「1 配置の 1 回の結果」は信用しない。

7. 結果

7.1 回避則の効果と方策との交互作用(20 配置、2×2)

障害物	歩行方策	回避	n	成功率	接触率	所要(成功分)
なし	既定(前進のみ)	なし / あり	5+5	0% / 0%	0% / 0%	—
なし	拡張	なし / あり	5+5	100% / 100%	0% / 0%	15.9 / 16.1 s
あり	既定(前進のみ)	なし / あり	15+15	0% / 0%	47% / **67%**	—
あり	拡張	なし / **あり(v1)**	15+15	13% / **93%**	87% / **0%**	16.0 / 33.6 s

回避則は拡張方策と組み合わせたときだけ効く。**既定方策に回避則を付けると接触率がむしろ 47% → 67% に上がった**。既定方策は停止指令には従わず約 0.3 m/s で前進し続ける一方、旋回指令にはわずかに追従するため、「減速して横へ逃げる」つもりの指令が減速なしの緩い旋回になり、経路脇の障害物へ寄っていく。回避則は「指令どおりに減速・旋回できる歩行方策」を前提にしており、前提が崩れると安全側に働かない。期待と異なる結果だがそのまま報告する。

7.2 45 配置 × 4 条件(拡張方策、障害物あり seed 5–49)

条件	成功率 [95% CI]	接触率	所要 [s]	経路効率
回避なし	4/45 = 9% [4–21]	91%	15.9	1.07
回避 v1	41/45 = 91% [79–96]	0%	31.6	1.00
回避 v2(コミット区間)	43/45 = 96% [85–99]	0%	31.7	0.81
v3(占有格子 + A*)	**45/45 = 100%** [92–100]	**0%**	**24.2**	**1.05**

転倒 0/135。所要・経路効率は成功分の平均。対応あり McNemar 正確検定: 回避なし vs v1 — 成功 $p < 0.0001$(v1 のみ成功 37、なしのみ 0)、接触 $p < 0.0001$(41 vs 0)。v1 vs v2 — 成功 $p = 0.69$(v2 のみ成功 4、v1 のみ 2)、所要時間の差 中央値 +1.1 s(符号検定 $p = 0.51$、Wilcoxon $p = 0.76$)。v3 vs v1 — 成功 $p = 0.125$(v3 のみ成功 4、v1 のみ 0)、所要時間(両方成功 41 配置)中央値 −3.4 s、v3 が速い 34 / 遅い 3(符号検定・Wilcoxon とも $p < 0.0001$)。v3 vs v2 — 成功 $p = 0.5$(v3 のみ 2、v2 のみ 0)、所要時間(43 配置)中央値 −6.1 s、36 / 5($p < 0.0001$)。v1・v2 の失敗 6 配置はすべて v3 で成功し、v3 で新たに失敗した配置はない。

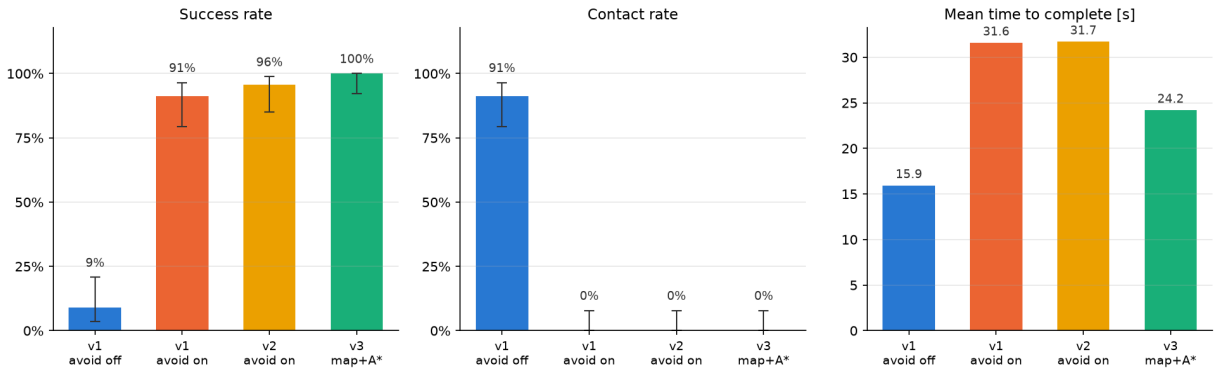


図 8: 45 配置 × 4 条件。左: 成功率、中: 接触率、右: 成功分の平均所要時間(誤差棒 = Wilson 95% CI)。

転倒 0/180。回避の有無による差は統計的に明確である。地図の有無(v3 と v1・v2)の差は、成功率では失敗数が少ないため示せないが、**所要時間の短縮は対応あり比較で明確**で、v3 は v1・v2 が時間切れになった 6 配置を含む全配置を 17〜46 s で完走した(§ 8)。

8. なぜ地図が必要か: 反応的回避との比較(アブレーション)

v3 の地図と経路計画がどれだけ効いているかを確かめるため、地図を持たない 2 つの手法を比較対象とした(図 9)。**v1** は進路上の障害物にその場で反応して減速・停止・旋回するだけの反応的回避で、v3 から地図と経路計画を取り除いたものにあたる(v3 でも下層の安全停止として使っている)。**v2** は v1 に「曲がる向きを 2 s 固定し、正面が空いたら 1 s 直進する」コミット区間を足したもので、地図なしの工夫でどこまで改善できるかを見るための比較である。

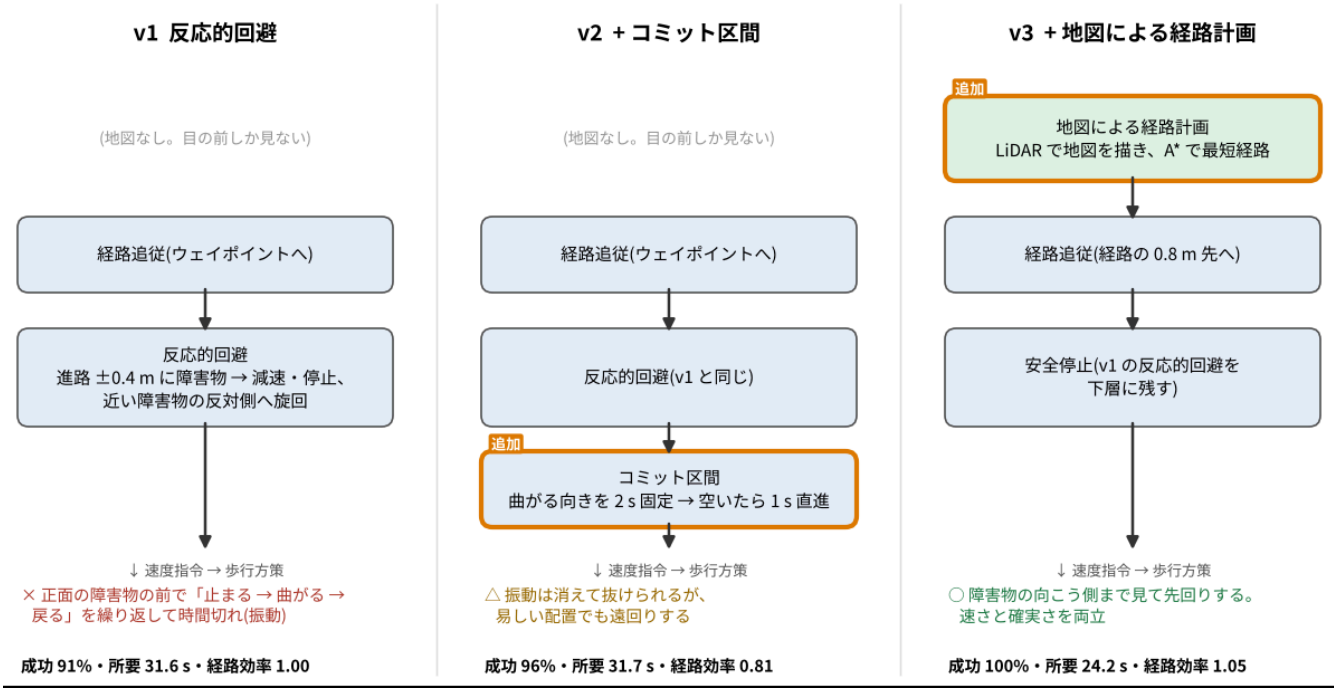


図 9: 比較した 3 つのナビゲーション層。前の手法に足した部分を橙の枠で示す。v1 は目の前の障害物にその場で反応するだけ、v2 は曲がる向きを一定時間固定する、v3 は地図を持って先回りする。数値は 45 配置での成功率、成功分の平均所要時間、経路効率。

地図を持たない 2 手法の失敗は、どちらも**障害物の向こう側を知らない**ことから来ている。v1 は経路の真正面に障害物があると「停止 → その場旋回 → 正面が空く → 経路追従が元の向きへ戻す → 再び停止」を繰り返し(振動)、接触も転倒もないまま時間切れになる。v1 の失敗 4 配置はすべてこの型だった。v2 はコミット区間で振動を止めて抜けられるようになるが、経路の脇にある障害物にもコミットして外側へ大回りするため、経路効率は 1.00 から 0.81 に下がり、平均所要時間も縮まらなかった(31.6 → 31.7 s、成功率の差は $p = 0.69$ で検定では示せない)。v3 は地図の上で障害物の内側を通る経路を先に引くため、どちらの失敗も起きない(図 10 右)。

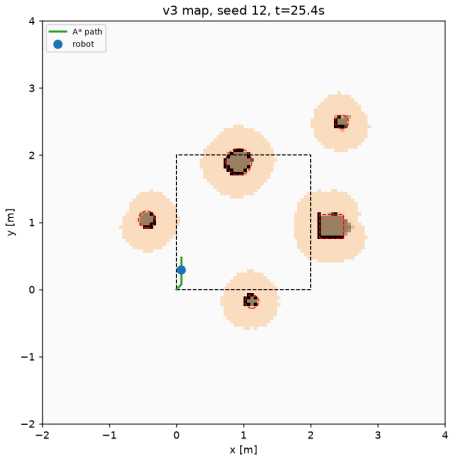


図 10 左: v3 が巡回中に構築した占有格子(seed 12、25.4 s 時点)。黒 = 占有、薄橙 = 膨張領域、赤破線 = 真の障害物。4 方向から見るため輪郭全体が占有になり、偽の占有はない。

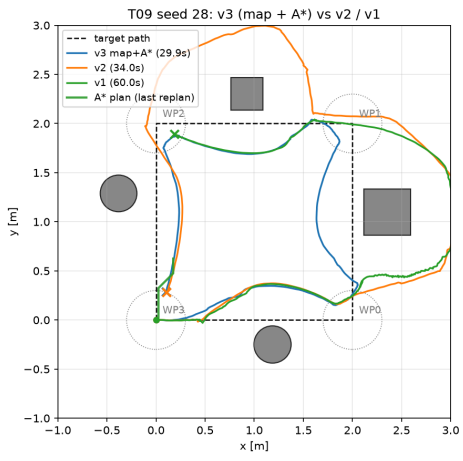


図 10 右: seed 28(経路上に障害物 4 個)。v1(緑)は振動して時間切れ、v2(橙)は外側へ大回り、v3(青)は計画経路に沿って障害物の内側を通り 29.9 s で完走。

経路を塞ぐ障害物の数で層別すると、地図の効果の大きさがはっきりする。反応的回避は障害物が増えるほど止まる・回る時間が増え、所要時間は v1 で 20.5 → 46.7 s、v2 で 24.5 → 41.6 s に伸びる。v3 は**すべての層で最も速く**、障害物が 3〜4 個でも 30 s で 1 周する。

経路上の障害物数	配置数	v1 所要 [s]	v2 所要 [s]	v3 所要 [s]	v3 成功
0	7	20.5	24.5	**19.1**	7/7
1	17	29.1	30.0	**21.5**	17/17
2	16	41.5	37.1	**27.4**	16/16
3〜4	5	46.7	41.6	**30.1**	5/5

所要は失敗 = 60 s を含む平均。v1・v2 が失敗した 6 配置は v3 ですべて成功し(うち seed 12, 25, 26, 28 は 2 回ずつ再実行して 8/8、接触 0)、v3 で新たに失敗した配置はない。v3 が v1 より 10 s 以上遅かった配置が 3 つあるが(いずれも成功)、下層の安全停止が働いて減速した回と考えられる。45 配置を通して A* が「経路なし」を返したことはなかった。地図は既知配置での単体テスト(占有セルが箱の正面 1.28〜1.33 m に出る、期待 1.30 m)で検証し、計算コストは地図更新 0.6 ms/step、A* 4 ms/回で、評価の実行時間は v1・v2 と変わらない。

まとめると、地図の効果は成功率よりも**所要時間**に強く表れる。成功率は v1・v2 でもすでに 91〜96% と高く、v3 との差は失敗数が少ないため検定では示せない(v1 比 $p = 0.125$、v2 比 $p = 0.5$)。一方、所要時間は対応のある比較で明確に短くなった(v1 比 中央値 −3.4 s、v2 比 −6.1 s、いずれも $p < 0.0001$)。

9. 3D LiDAR: 高さ方向で通行可否が変わる障害物

ここまでの障害物はすべて高さ 0.6 m 以上で、センサ面(床から 0.44 m)の 2D スキャンに必ず映る。実環境で問題になるのは、床に置かれた低い物や机の天板のように**スキャン面に映らないが胴体には当たる**障害物である。立位と歩行中の実測(胴体最高点 0.35 m、歩行中 0.39 m、遊脚の最大高さ 0.02 m)から、**低い箱**(高さ 0.15 m、スキャン面の 29 cm 下、遊脚より上なので踏むと転倒しうる)と**テーブル**(天板 0.32〜0.36 m、スキャン面より下で胴体より低い。脚は半径 2 cm の円柱 4 本)を設計した。2D スキャンでは低い箱は見えず、テーブルは細い脚しか見えないため「通れる」と判断される。

3D 化はナビゲーション層を変えずに入力だけを差し替える形にした。Raycaster を 72 方位 × 16 層(垂直 ±30°、4° 刻み)にして点群を得、ヒット点の高さで「地面(< 0.05 m)/ 胴体の高さ帯(0.05〜0.49 m)/ 頭上 / 自機(下向きのレイが自分の脚に当たる点。胴体フレームの矩形で除外)」に分類し、胴体帯の点の最小水平距離を方位ごとに 72 本の仮想スキャンへ投影して、v1 の下層と v3 の地図にそのまま渡す。追加コストは +0.5〜0.8 ms/step で、評価・録画の速度にほぼ影響しない。

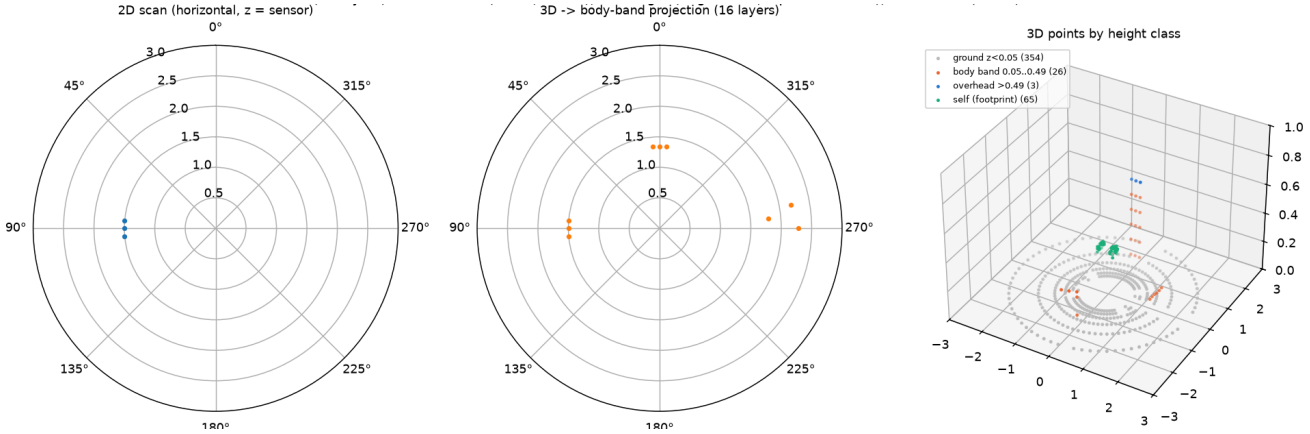


図 11: 既知配置(正面 1.3 m に低い箱、右に縁 1.5 m のテーブル、左 1.5 m に高い箱)での単体テスト。左: 2D スキャンには高い箱しか映らない。中: 3D の高さ帯投影では 3 つとも映る。右: 点群の高さ分類(灰 = 地面、橙 = 胴体帯、青 = 頭上、緑 = 自機)。

低い箱 1〜2 個・テーブル 1 台・高い障害物 1〜2 個を乱数配置し、少なくとも 1 つの低い障害物が経路上に来る **30 配置**(seed 100–129)で、v3 の入力だけを変えて比較した。

v3 の入力	成功率 [95% CI]	接触率	転倒	所要 [s]	失敗の内訳
2D LiDAR	1/30 = 3% [1–17]	97%	6	16.6	低い箱に接触 7(うち転倒 6)、天板に接触 23
3D 16 層 ±30°(採用)	**30/30 = 100%** [89–100]	**0%**	0	27.5	—
3D 25 層 ±12°(1° 刻み)	10/30 = 33% [19–51]	67%	18	24.8	低い箱に接触 20(うち転倒 18)
3D 30 層(±12° 1° 刻み + 下向き 5 層)	29/30 = 97% [83–99]	0%	0	30.6	時間切れ 1

所要は成功分の平均。2D vs 3D 16 層: 成功・接触とも McNemar $p < 0.0001$(不一致 29 / 0)。25 層 vs 16 層: $p < 0.0001$(20 / 0)。30 層 vs 16 層: 成功 $p = 1.0$、所要は 16 層が速い 8 / 遅い 2(符号検定 $p = 0.11$)。

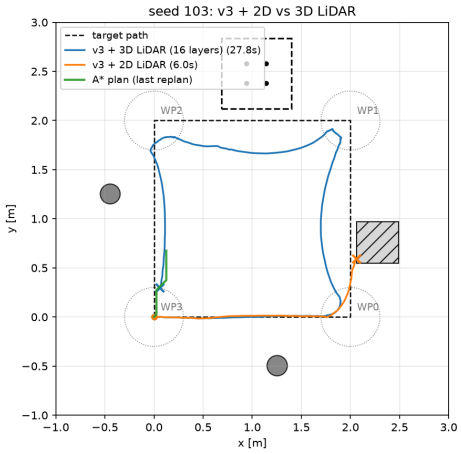


図 12 左: seed 103。2D(橙)は右の低い箱(斜線)を踏んで 6 s で転倒。3D(青)は低い箱と上のテーブル(破線)を避けて 27.8 s で完走。

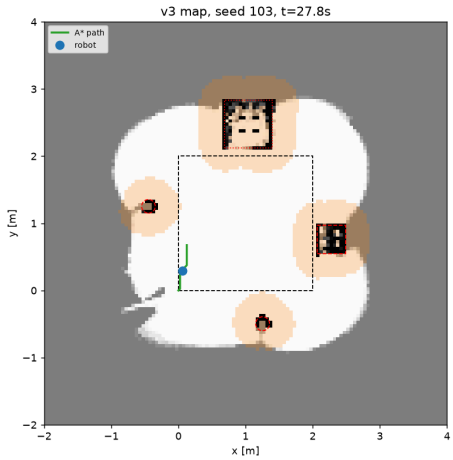


図 12 右: 3D で構築した占有格子。低い箱とテーブルの天板が占有(黒)になり、膨張領域(薄橙)で経路から外れる。

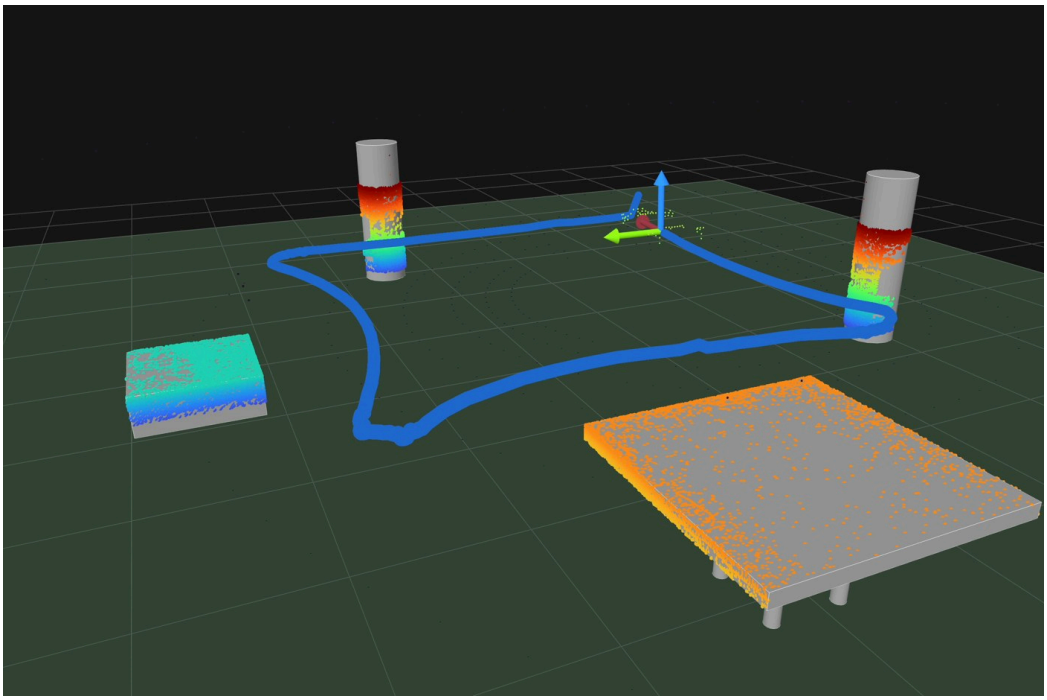


図 13: seed 103 の 3D 版の巡回ログ(MCAP)を Foxglove で表示(終了時)。胴体の高さ帯の点を高さで色分けして積算すると、低い箱の上面(水色)、テーブルの天板(橙)、円柱の側面(高さ方向に青 → 赤)が浮かび上がる。青線は軌跡、座標軸はロボットの現在位置。動画ではこの点群が巡回に伴って積み上がる様子を示した。

結果から 2 つの知見が得られた。第一に、**層の密度より垂直視野、特に下方の視野が効く**。テーブルの縁の検出が 4° 刻みの層の粗さで 0.3 m 遅れることを直すために ±12°・1° 刻みの 25 層を試したところ、縁の誤差は 0.37 m → 0.03 m(90 パーセンタイル)に下がったが、最下層(−12°)が高さ 0.15 m の箱に届くのは 1.35 m より遠い場合だけで、それより近づくと箱が全レイの下に隠れ、空き更新で地図から消えて踏んだ(成功 33%)。下向きの粗い層を足すと接触は 0 に戻った。センサ選定では角度分解能より「足元まで見える垂直視野」を先に確保すべきことを示している。第二に、**センサを正確にしても速くならない**。30 層は縁を連続して捉えるが、WP1 の 0.6 m 隣にテーブルがある配置で、v3 の経路(膨張 0.35 m)が縁から 0.35〜0.45 m を通ると、v1 下層の回廊(±0.4 m)と停止距離(0.45 m)が反応してその場旋回を続け、所要が 55〜60 s に伸びた(16 層は縁の検出が間欠的で、たまたま抜けられる)。原因は計画層と下層の安全余裕の干渉と、ウェイポイントが障害物に近すぎることで、実機の巡回でウェイポイントを壁際に置いたときに起きる現象と同じである。既定は 16 層とし、25 層・30 層は試作として残した。

10. 工夫した点

1. **層構造の設計**: 学習するのは歩行だけ。ナビゲーションは決定的で説明可能なため、失敗の原因をログから特定して設計を直せる(§5 の回避規則、§8 の地図の導入、§9 の 3D 化はいずれもログの診断から導いた)。v3 で実機 Go2 と同じ「自己位置 → 地図・計画 → 速度指令 → 歩行」の層が揃い、比較対象の v1・v2 も `--avoid-version` で切り替えて同じ条件で評価できる。
2. **指令範囲の拡張**: 例題の「前進のみ」の限界を 1 配置で実証し、再学習の必要性を 20 配置で確認。
3. **地面除去と単体検証**: センサを幾何期待値・Metal/CPU 一致・歩行中の偽ヒット 0 本で検証してから制御に使った。
4. **評価設計**: 衝突判定の欠陥(LiDAR では正面押し付けを検出できない)を実測で見つけて主指標を変更。対応あり検定と信頼区間、非決定性の切り分け。「期待と違う結果」(既定方策 + 回避則で接触率が上がる、v2 が平均では改善しない、v3 の成功率の差は検定で示せない)をそのまま報告。
5. **3D 化の設計**: 点群を高さ帯で 2D に投影する前処理だけを差し替え、地図・計画・歩行は無変更。障害物の寸法は推測でなく立位・歩行中の実測(胴体最高点・遊脚高さ)から決めた。仕様どおりの試作(25 層)が失敗した原因を幾何で説明し、採用構成を数値で選んだ。
6. **計算資源**: Mac 1 台で完結。学習は Metal(4096 環境で 7 万 steps/s、300 it = 9 分)、1 環境の評価は CPU(Metal より 10 倍速いことを実測して切替)。2D/3D の評価(計 300 エピソード)も同じ環境で完結。
7. **可視化**: ROS なしで MCAP を書き出し、Foxglove で点群・軌跡・指令を実機と同じ形で確認できるようにした。
8. **開発プロセス**: タスク仕様(受け入れ条件つき)→ 実装・計測 → 報告 → レビュー、の反復を 13 回。各報告はリポジトリの `reports/` に残し、設計変更の経緯を追跡できる。

11. 限界と今後

- **真の姿勢を使用**: 自己位置推定(SLAM)は実装していない。v3 の地図更新・A* の始点・ローカル目標の追従はすべてシムの真の姿勢を使うため、本結果は「自己位置が正確なら planner 層はこれで足りる」という上限である。実機では LiDAR SLAM / オドメトリの誤差が地図に累積するので、その影響は未評価。
- **地図と計画の簡略化**: 静止障害物前提の log-odds(動く障害物は消えるまで時間がかかる)、未知セル = 通行可、A* は経路長のみ最小化(膨張領域の外なら障害物のすぐ脇を通る。実機 costmap のような距離に応じたコストは未導入)。障害物の間隔が 1.2 m 以上という生成条件のため「経路なし → 膨張縮小 → 直線」の退化経路は未検証。
- **静止障害物・平坦地形のみ**。動く障害物、センサノイズ、ドメインランダム化は未対応。
- **シミュレータの非決定性**: CPU バックエンドの並列計算に由来すると考えられるが発生源は未特定。評価は配置数で平均化して対処した。

- **3D の範囲**: 低い障害物と張り出し物(天板)は 3D で扱ったが、崖・段差(床の不連続)と斜面は未対応。斜面・段差の踏破は歩行方策の不整地での再学習が必要で、センサの問題ではない。3D の層の粗さによる天板の縁の検出遅れ(0.3 m)は残っている。
- **計画層と下層の干渉**: v3 の膨張半径と v1 下層の回廊・停止距離が独立に設定されており、狭所(ウェイポイントの近くの障害物)で下層が計画を妨げる。下層の判定を「計画経路上の障害物」に限るのが次の改良。
- **SLAM の素材**: 3D 版の MCAP には全点群・IMU(角速度・加速度)・真の姿勢(TF)を記録しており、LiDAR-慣性オドメトリを後から評価する正解付きデータとして使える。

12. 再現と使用 OSS

環境: macOS(Apple Silicon)、Python 3.11、genesis-world 1.4.4、rsl-rl-lib 5.5、torch 2.14、foxglove-sdk 0.30。`pip install -r requirements.txt` の後、`python go2_train_patrol.py -e go2-patrol -b 4096 --max-iterations 300`(学習)、`python go2_patrol.py -e go2-patrol --ckpt 299 --seed 10 --avoid --avoid-version v3 --backend cpu --out out.mp4 --mcap out.mcap`(巡回・録画。`--avoid-version v1|v2|v3`、`--lidar 2d|3d`。3D の混合配置は seed 100 〜 129)、`python eval/run_eval.py ... && python eval/summarize.py`(評価)。詳細はリポジトリの README と `reports/01〜13`。使用 OSS: Genesis(Apache-2.0、Go2 例題 `go2_env.py` / `go2_train.py` を改変・複製し NOTICE に明記)、rsl_rl(BSD-3)、PyTorch、Foxglove SDK(MIT)。本リポジトリは Apache-2.0。